Network Threat Detection

Reverse Shell & C2 Beaconing — Packet-Level Analysis and Malicious/Benign Traffic Discrimination

---

Section 1: Executive Summary

This project analyzes two core post-exploitation network techniques — an interactive reverse shell and periodic C2 beaconing — entirely from raw packet capture, without relying on endpoint telemetry (Sysmon) or a SIEM. This is a deliberately different analytical skill from Projects 1 and 2: rather than correlating log events, the evidence here is read directly from network traffic in Wireshark.

Consistent with the discrimination-focused methodology used in Project 2's rule tuning, each technique was captured twice: once as a genuine malicious simulation (true positive), and once as a carefully designed benign lookalike (false positive) — a legitimate remote-support check-in for the reverse shell comparison, and a jittered software update-checker for the beaconing comparison. The objective in each case was not merely to demonstrate the malicious technique, but to identify the specific, articulable packet-level indicators that separate it from traffic that looks superficially similar.

Both true-positive captures were independently corroborated across multiple data sources — the PowerShell script's own execution log, the Kali-side listener/server log, and the Wireshark packet capture — which agreed closely in every case, providing high confidence in the accuracy of the analysis.

Section 2: Environment & Methodology

Lab environment: the same isolated lab used in Projects 1 and 2 — a Kali Linux attacker VM and a Windows 10 victim VM on a sealed VirtualBox NAT Network (SOC-Lab-Net), fully rebuilt from scratch prior to this project.

Capture point: Wireshark was run on the Kali VM's eth0 interface. Since Kali hosts the listener/server in every scenario (the netcat listener for the reverse shell tests, the HTTP server for the beaconing tests), capturing at this single point records the complete bidirectional conversation for every test — both VMs are the only two parties in each connection, so nothing is missed by capturing at one end.

Testing methodology — atomic, paired testing: consistent with Project 2, each technique was tested as an isolated true-positive/false-positive pair rather than a single capture, so that the analysis produces a genuine discrimination case rather than just a demonstration of the malicious technique in isolation.

Workflow per technique:

1. Start listener/server on Kali
2. Start Wireshark capture on the relevant port, confirm 0 packets baseline
3. Trigger the true-positive script on Windows, capture and save
4. Repeat with the false-positive (benign lookalike) script
5. Compare packet counts, session structure, and (for beaconing) timing regularity between the two captures

---

Section 3: Reverse Shell Analysis

MITRE ATT&CK Mapping: T1059.001 (Command and Scripting Interpreter: PowerShell), T1095 (Non-Application Layer Protocol — raw TCP, not wrapped in HTTP/DNS/etc.)

3.1 True Positive

Test script: a PowerShell reverse shell establishing a raw TCP connection to a Kali-hosted listener on port 4444, redirecting cmd.exe's standard input/output/error through the connection to produce a fully interactive remote shell.

Debugging note — a genuine troubleshooting exercise: the initial version of this script connected successfully but was silently non-interactive — commands typed on the attacker side produced no output, despite Wireshark confirming data was reaching the victim machine. Systematic debugging ruled out several candidate causes in turn (confirming the network-read-to-stdin loop existed, confirming a newline was appended via WriteLine()) before identifying a missing flush on the stdin writer as a first fix. When the identical symptom persisted after that fix, further diagnostic steps covering the process's I/O redirection configuration (UseShellExecute, stream targeting, encoding) were applied before the shell became genuinely interactive. This mirrors the kind of layered, elimination-based debugging real reverse-shell and C2 tooling issues require in practice.

Session activity: once corrected, an interactive session was conducted — whoami, hostname, and ipconfig were each typed on the attacker side and returned live output from the victim, followed by exit.

Results:

- **67 packets captured** over the session (19 shown after applying the tcp.port == 4444 filter — the remainder is unrelated background broadcast/ARP traffic)
- **Session duration: 5 minutes 54 seconds**

- **Follow TCP Stream:** 4 client packets, 6 server packets, **8 turns** — a clear back-and-forth conversational pattern
- Raw stream content confirms full command/response pairs:
- whoami → kushwanth\kushwhostname → kushwanthipconfig → [full Windows IP Configuration output]exit

Evidence (Appendix): Fig. 1–7, reverse shell true positive

3.2 False Positive (Benign Lookalike)

Test script: a PowerShell script simulating a legitimate remote-support/monitoring agent — connects to the same Kali listener, sends a single structured JSON status report, and closes the connection without any further exchange.

Payload sent:

{"uptime":"0 days, 0 hours","agent":"CorporateHelpdeskAgent","timestamp":"2026-08-22 14:25:30","hostname":"KUSHWANTH","status":"OK","os":"Microsoft Windows 11 Home"}

Results:

- **8 packets total** (SYN / SYN-ACK / ACK → one PSH-ACK data push → ACK → FIN-ACK / FIN-ACK → ACK)
- **Session duration: 25 seconds**
- **Follow TCP Stream:** 1 client packet, 0 server packets, **0 turns** — a one-way push with no conversational exchange

Evidence (Appendix): Fig. 1–5, reverse shell false positive

3.3 Discrimination Summary — Reverse Shell

Indicator	True Positive	False Positive
Total packets	67	8
Session duration	5m 54s	25s
Conversational turns	8	0
Traffic pattern	Sustained, bidirectional, human-paced (typed commands + returned output)	Single push, immediate close

Indicator	True Positive	False Positive
Content	Live command output (system info, network config)	One static structured status report

Key takeaway: packet/byte count alone is a weak signal — what reliably separates these two sessions is the **presence of sustained, bidirectional turn-taking** after the initial handshake. A connection that immediately pushes one payload and closes is structurally different from one that stays open and exchanges data repeatedly, regardless of how "suspicious" the destination port might look in isolation.

Section 4: Beaconing Analysis

MITRE ATT&CK Mapping: T1071.001 (Application Layer Protocol: Web Protocols)

4.1 True Positive

Test script: simulates C2 beaconing — 20 HTTP GET requests sent to a Kali-hosted HTTP server on port 8080 at a fixed 30-second interval, each including a static session identifier (?id=a1b2c3d4-session&seq=N).

Corroboration across three independent sources:

- **PowerShell script log:** 20 beacons logged, 11:05:40 → 11:15:13, consistent ~30s cadence
- **Kali HTTP server log:** 20 GET requests received with matching session-ID sequence numbers
- **Wireshark packet capture:** SYN packets isolated and measured directly

Measured interval (16 consecutive gaps, from SYN timestamps):

Gap #	Interval (s)
1–4	30.115, 30.110, 30.122, 30.141
5–8	30.087, 30.114, 30.172, 30.167
9–12	30.145, 30.100, 30.112, 30.095
13–16	30.158, 30.133, 30.110, (avg below)

Average interval: 30.13 seconds. Range: 30.09–30.17s — a spread of under 0.1 second. This tight, near-mechanical consistency is the defining signature of automated beaconing.

Evidence (Appendix): Fig. 1–7, beaconing true positive (includes PowerShell log and Kali server log as supporting exhibits)

4.2 False Positive (Benign Lookalike)

Test script: simulates a legitimate software update-checker — HTTP GET requests to the same server, but on a **randomized/jittered** 30–45 second interval rather than a fixed cadence, with a realistic request path (/?version=3.1.4&arch=x64) resembling genuine update-check telemetry.

Corroboration:

- **PowerShell script log:** 16 checks completed, each logging its own randomized "next check" delay (37s, 44s, 30s, 44s, 42s, 44s, 42s, 40s, 34s, 45s, 37s, 45s, 42s, 33s, 33s, 41s)
- **Kali HTTP server log:** matching GET requests, same irregular timing
- **Wireshark packet capture:** SYN-to-SYN gaps measured directly, matching the script log almost exactly (e.g., 37.11s, 44.04s, 30.04s, 44.03s, 42.04s…)

Measured interval: average ~39.6 seconds, range 30–45 seconds — a spread of 15 seconds, roughly 150x wider than the true positive's variance.

Evidence (Appendix): Fig. 1–7, beaconing false positive

4.3 Discrimination Summary — Beaconing

Indicator	True Positive	False Positive
Average interval	30.13s	39.6s
Interval range	30.09–30.17s	30–45s
Interval variance	< 0.1s	~15s
IO Graph pattern	Sharp, evenly-spaced, near-identical spikes	Visibly uneven spacing between spikes
Request path	Static session ID, no version/context info	Realistic version/architecture query string

Key takeaway: for beaconing specifically, **timing regularity is the discriminating signal, not the presence of periodic traffic itself** — both captures show a host repeatedly contacting the same server at roughly 30-45 second intervals, which could look identical to a coarse "is this host talking to the same destination repeatedly" rule.

Only measuring the actual variance between intervals reveals the difference: real automated C2 beacons tend toward mechanically tight timing, while legitimate polling clients are commonly built with deliberate jitter specifically to avoid this kind of pattern-based detection — meaning a detection rule keying purely on "periodic HTTP requests" would false-positive constantly against ordinary background software, and a rule keying on interval *tightness* is far more precise.

Section 5: Conclusion

This project demonstrated network-layer threat detection as a distinct discipline from the endpoint-log analysis in Projects 1 and 2 — reading attacker behavior directly from packet captures rather than through Sysmon/Splunk. Two techniques were analyzed, each with a paired true-positive/false-positive comparison consistent with the discrimination-focused methodology established in Project 2's detection rule tuning.

For the reverse shell, the reliable discriminator was **conversational structure** — sustained bidirectional exchange versus a single one-way push. For beaconing, packet-level presence alone was insufficient; the reliable discriminator required **quantifying interval variance** — a beacon's near-mechanical timing precision versus a legitimate client's deliberate jitter. Both findings reinforce a theme carried across all three portfolio projects: effective detection consistently comes from precisely characterizing *how* activity behaves (timing, structure, conversational pattern) rather than relying on surface-level indicators (a port number, the mere presence of periodic traffic) that legitimate activity can just as easily produce.

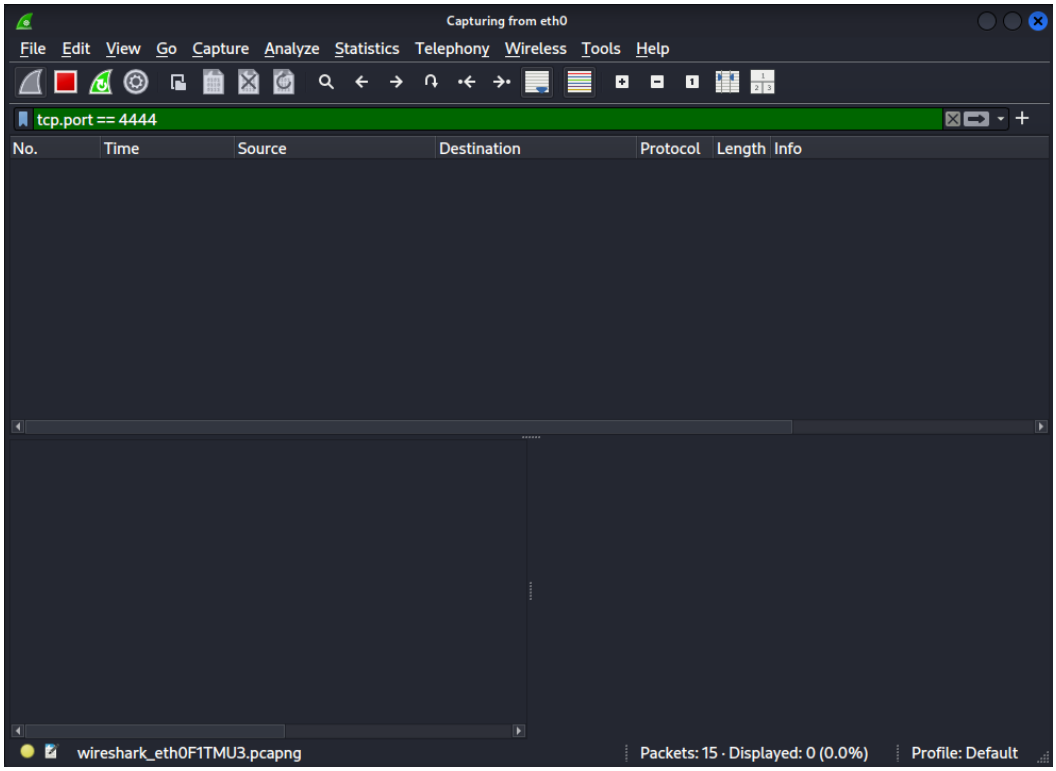
The reverse-shell true positive also involved genuine, multi-step debugging of a non-interactive shell — tracing a silent failure through several candidate causes before resolving it — reflecting the kind of layered troubleshooting real offensive and defensive tooling work regularly requires.

Appendix — Evidence Screenshots

Format note for Word: image directly above its caption, one figure per block, matching Projects 1 and 2's appendix format.

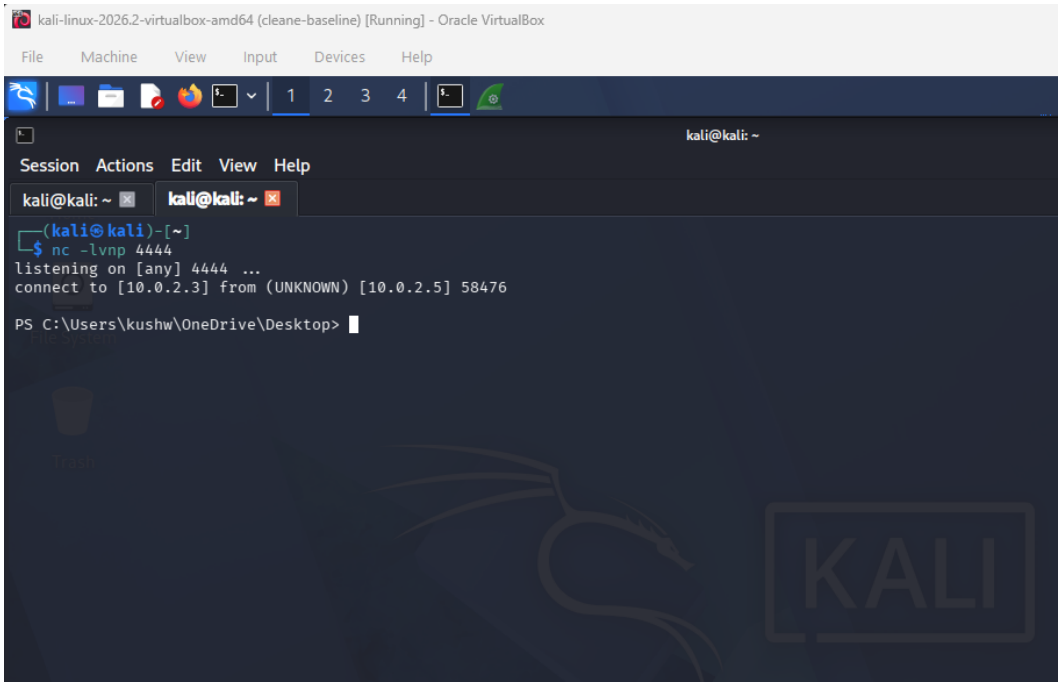
Reverse Shell — True Positive

Fig. 1 — Baseline capture, 0 packets



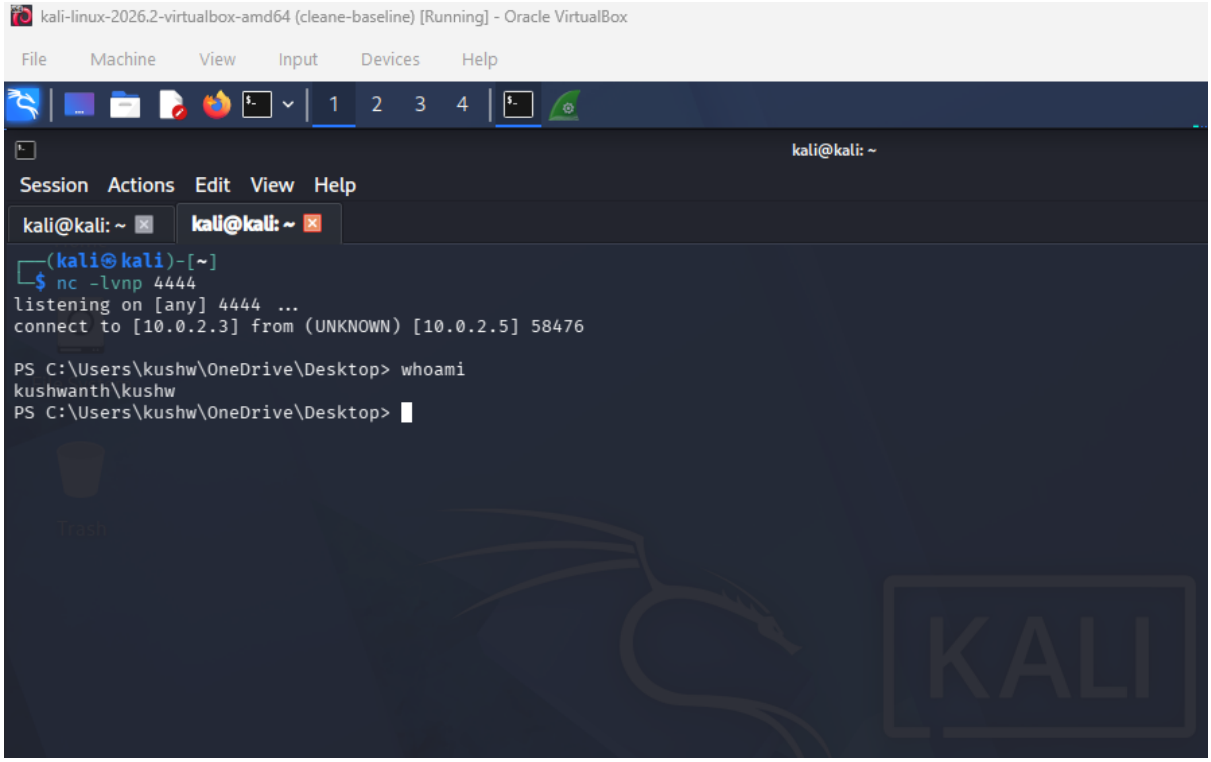
Wireshark capture running on eth0, filter tcp.port == 4444 applied, before the script was triggered.

Fig. 2 — Netcat connection established



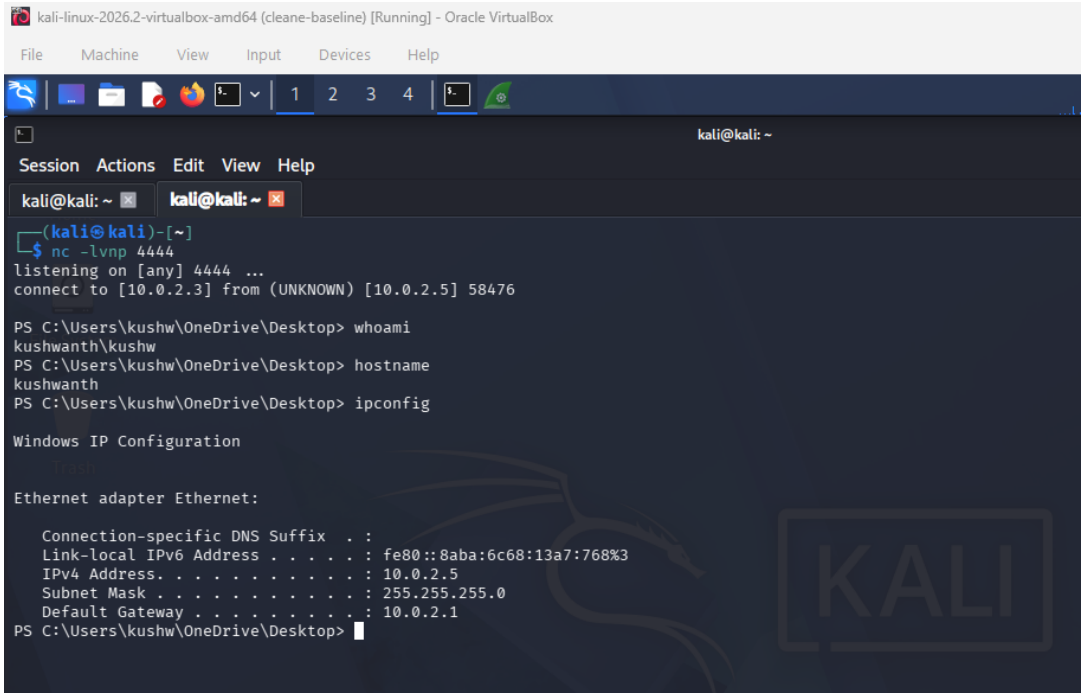
The Kali netcat listener showing the incoming connection from the Windows victim.

Fig. 3 — Interactive command confirmed (whoami)



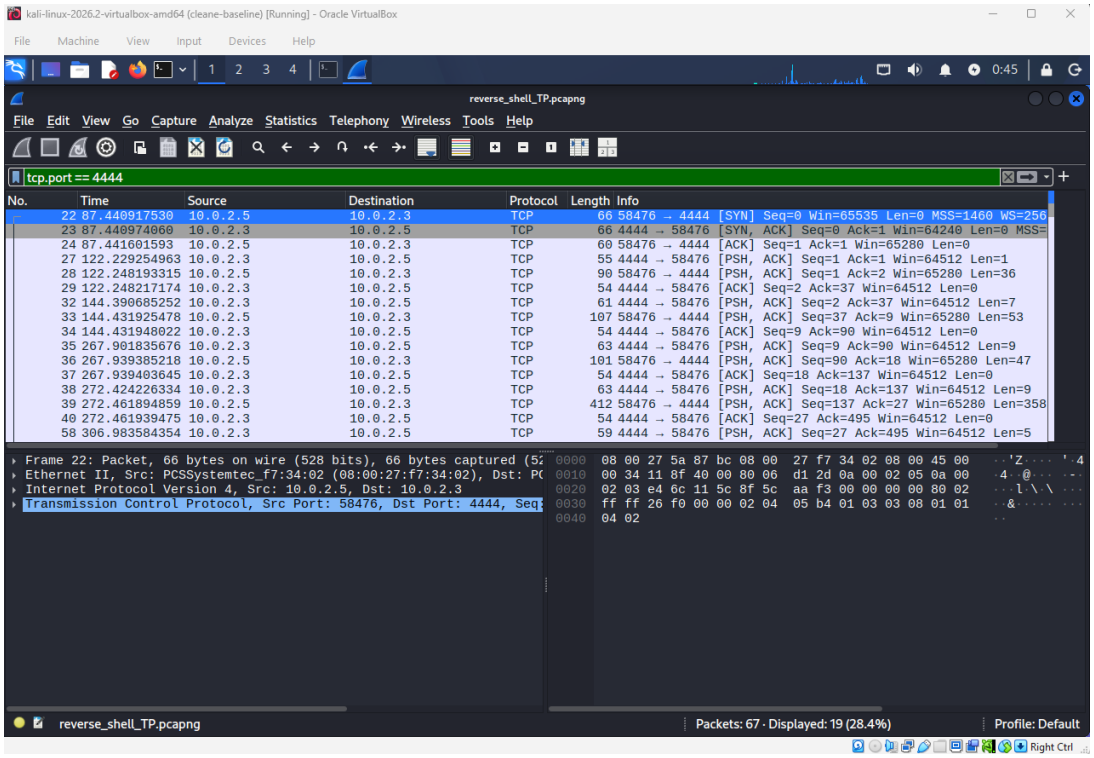
The netcat terminal after the redirection fix, showing whoami returning live output — confirms the shell is genuinely interactive.

Fig. 4 — Extended interactive session



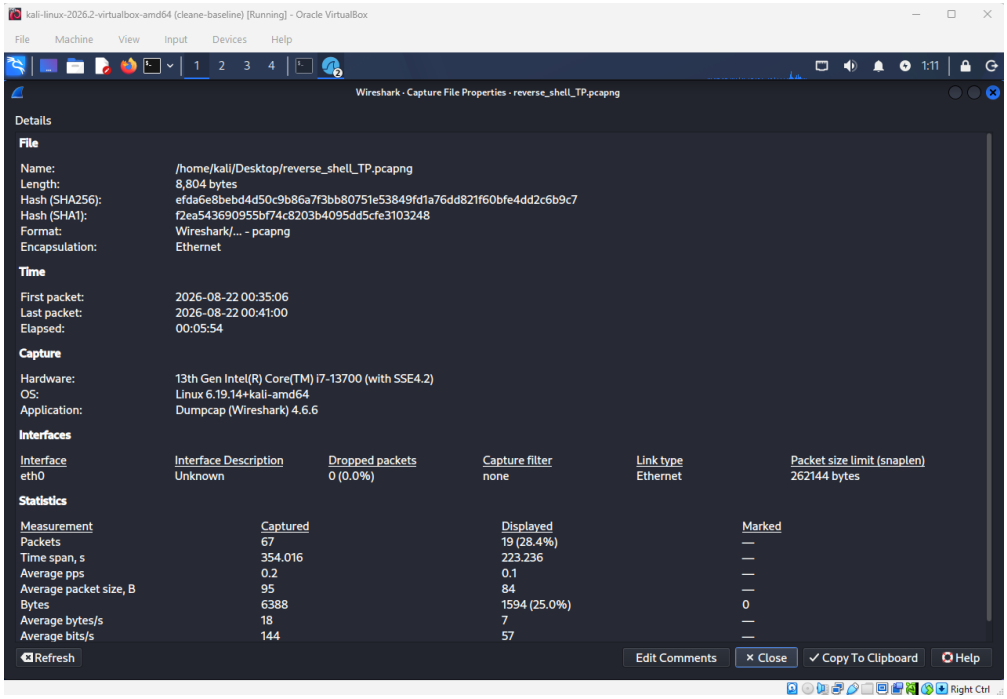
hostname and ipconfig executed in sequence, each returning live output, demonstrating a sustained interactive session.

Fig. 5 — Full packet list



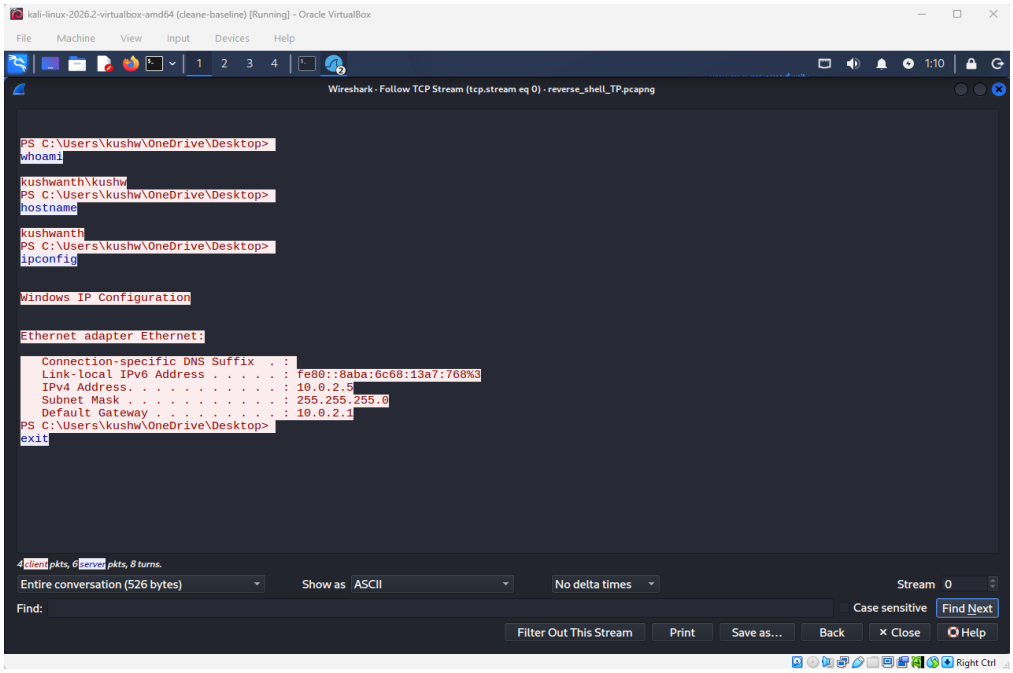
The complete Wireshark capture for this session — 67 packets captured, 19 shown after filtering.

Fig. 6 — Capture file properties



Session statistics: 67 packets, 8,804 bytes, 5 minutes 54 seconds elapsed.

Fig. 7 — Follow TCP Stream



The reconstructed conversation: 4 client packets, 6 server packets, 8 turns — full command/response text visible.

Reverse Shell — False Positive

Fig. 1 — Baseline capture, 0 packets

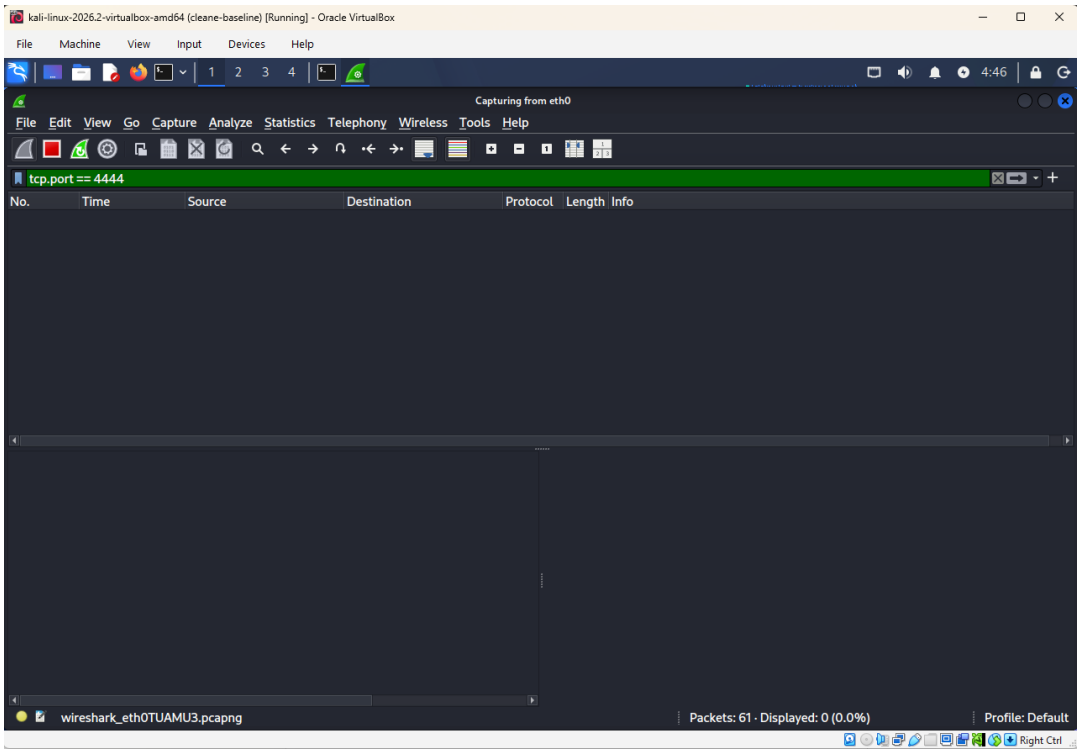
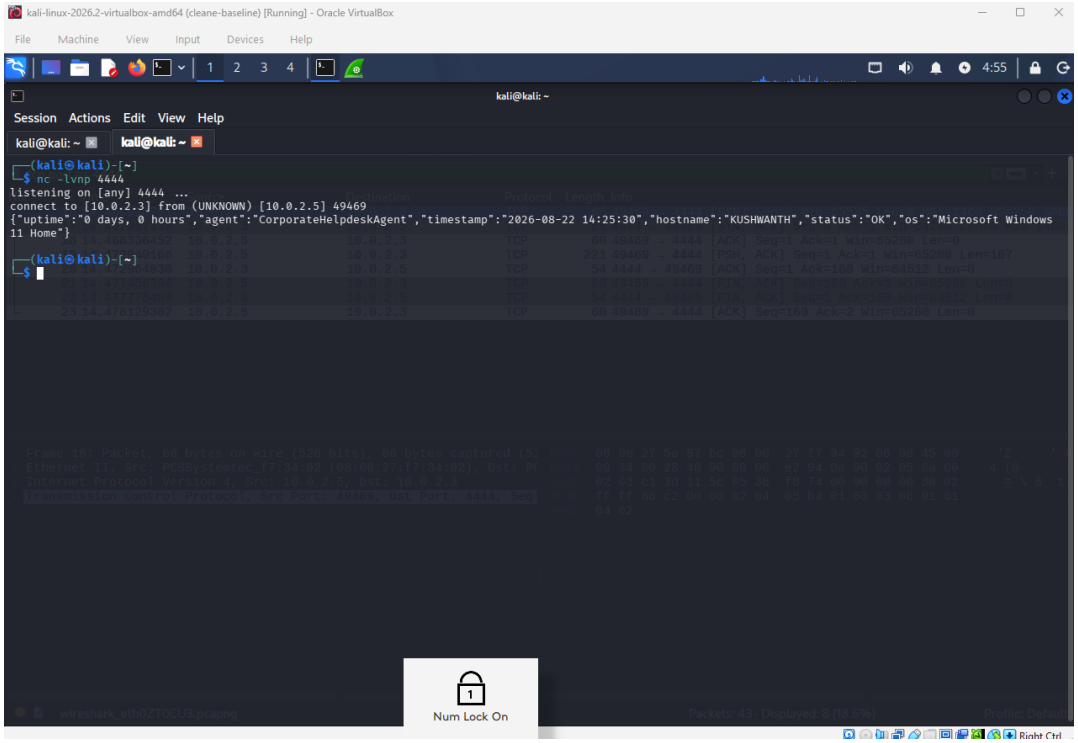
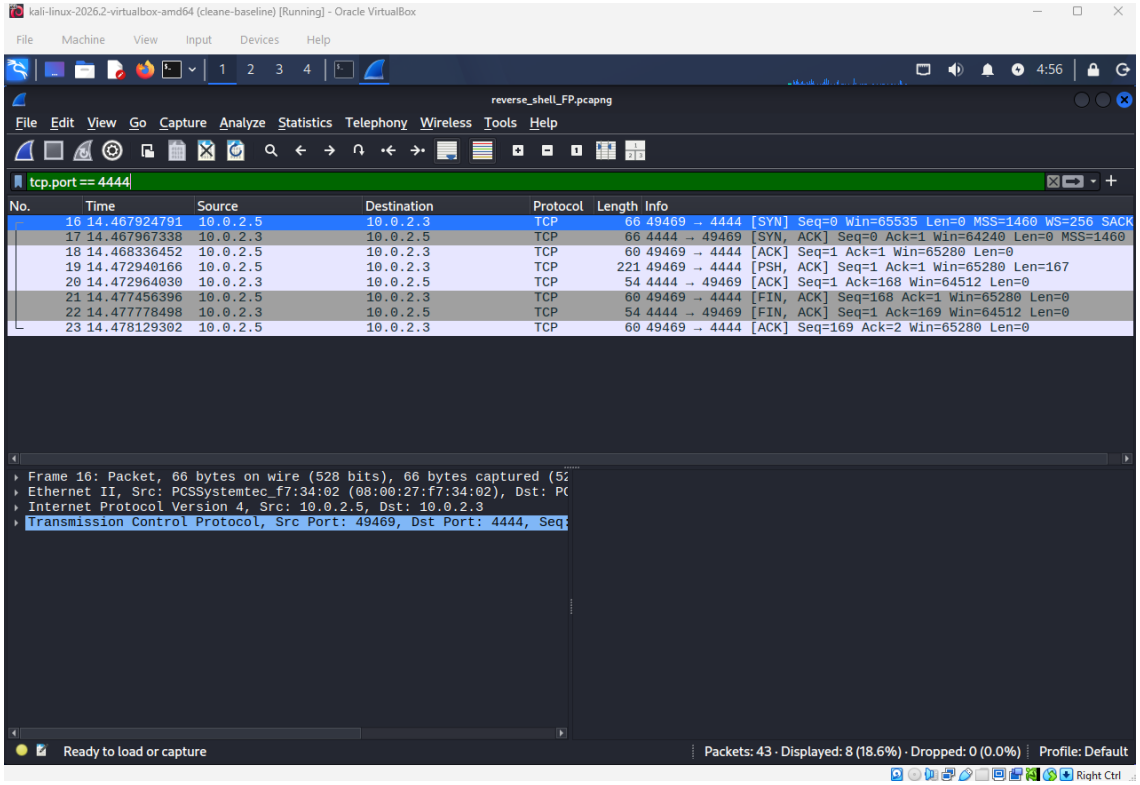


Fig. 2 — Netcat terminal: connection and status report received



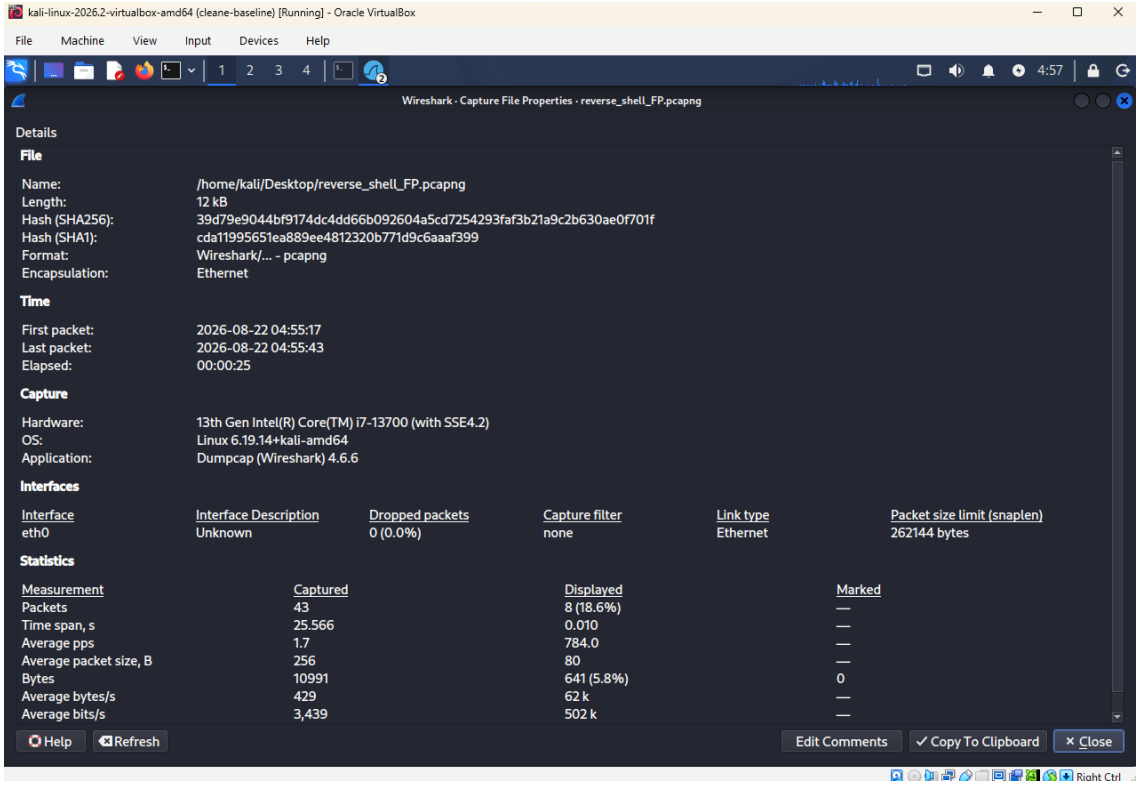
The single structured JSON status report received in full, connection closing on its own shortly after.

Fig. 3 — Full packet list



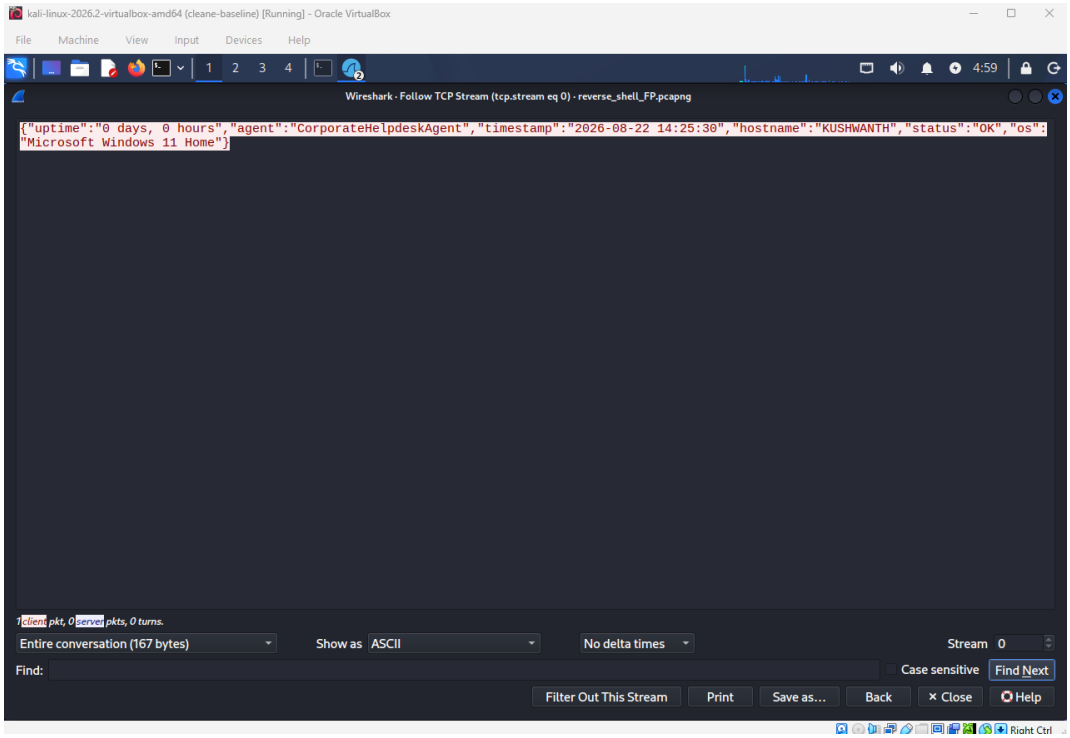
8 packets total: handshake, one data push, clean close.

Fig. 4 — Capture file properties



12 kB, 25-second session duration.

Fig. 5 — Follow TCP Stream



1 client packet, 0 server packets, 0 turns — the single JSON payload, no further exchange.

Beaconing — True Positive

Fig. 1 — Baseline capture, 0 packets

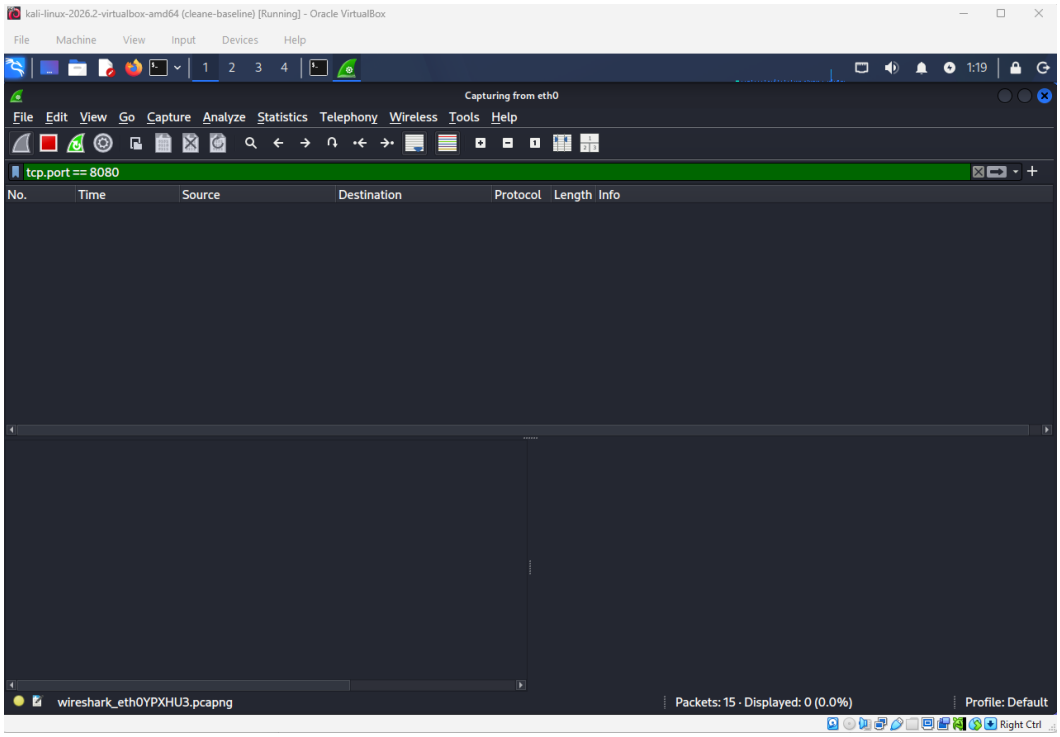
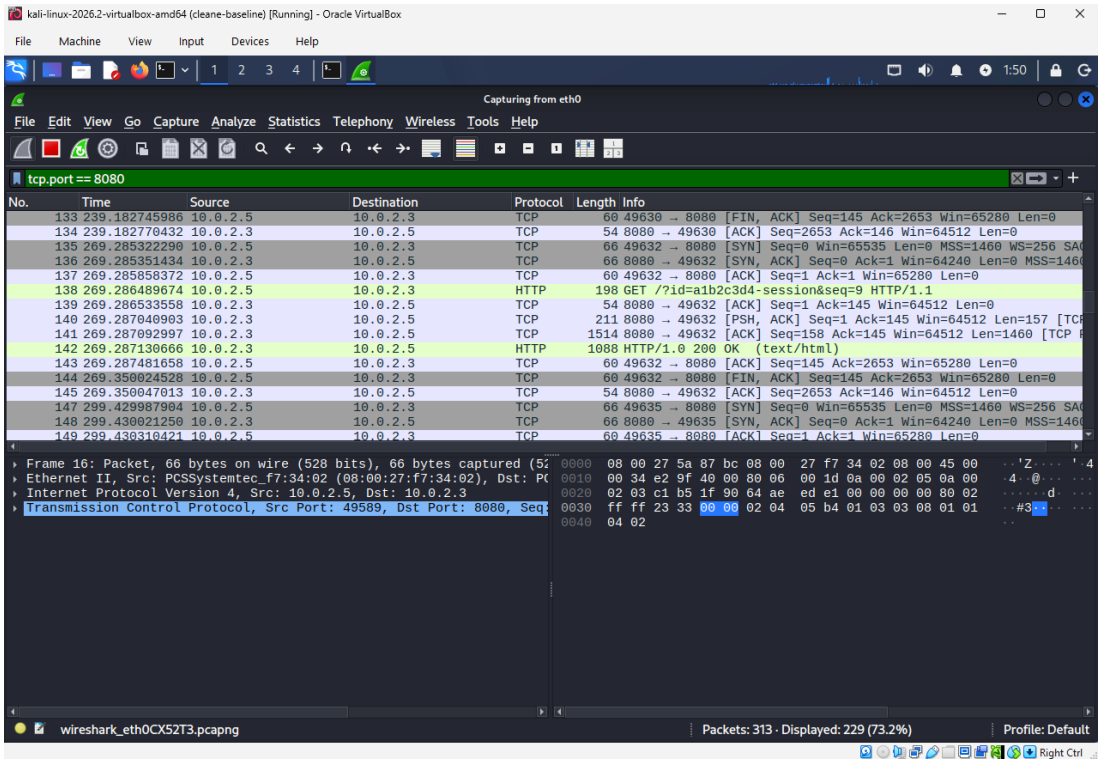
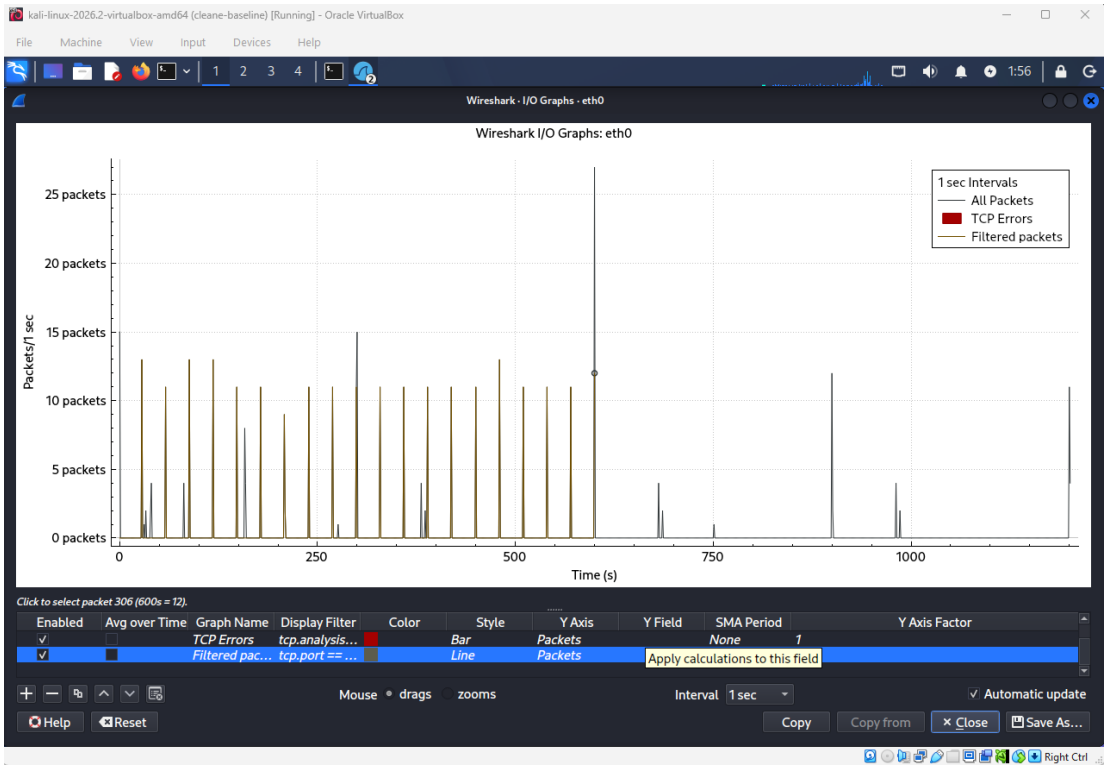


Fig. 2 — Full packet list



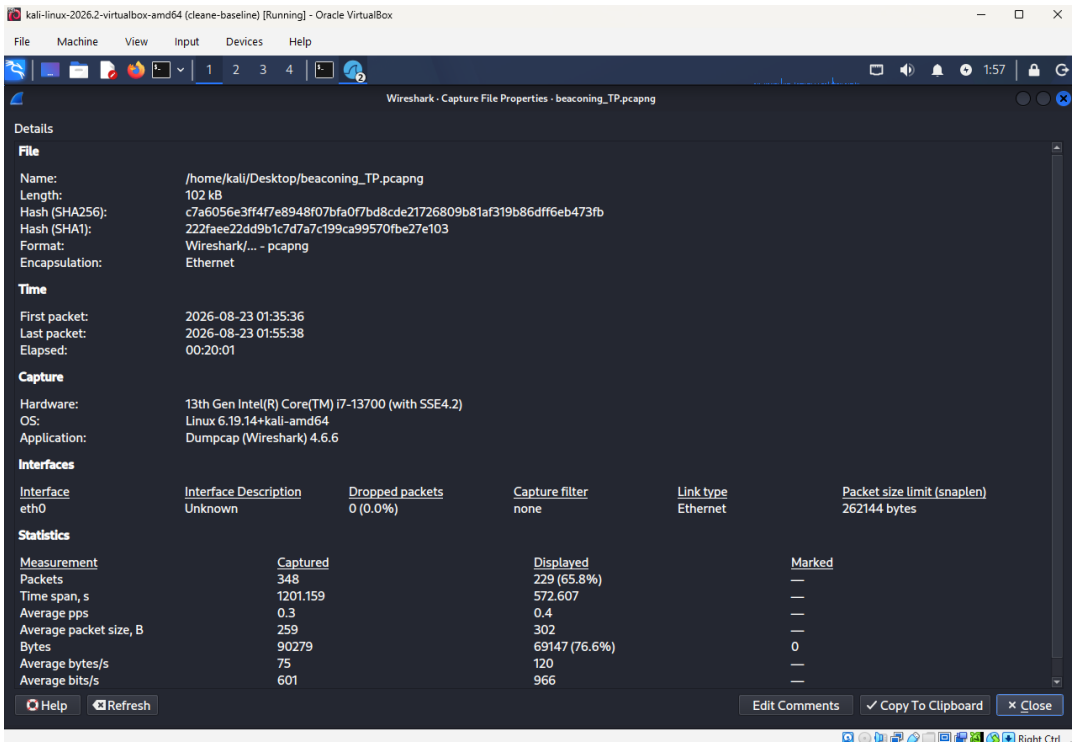
Repeating connection pattern visible down the packet list at ~30-second intervals.

Fig. 3 — IO Graph, 1-second intervals



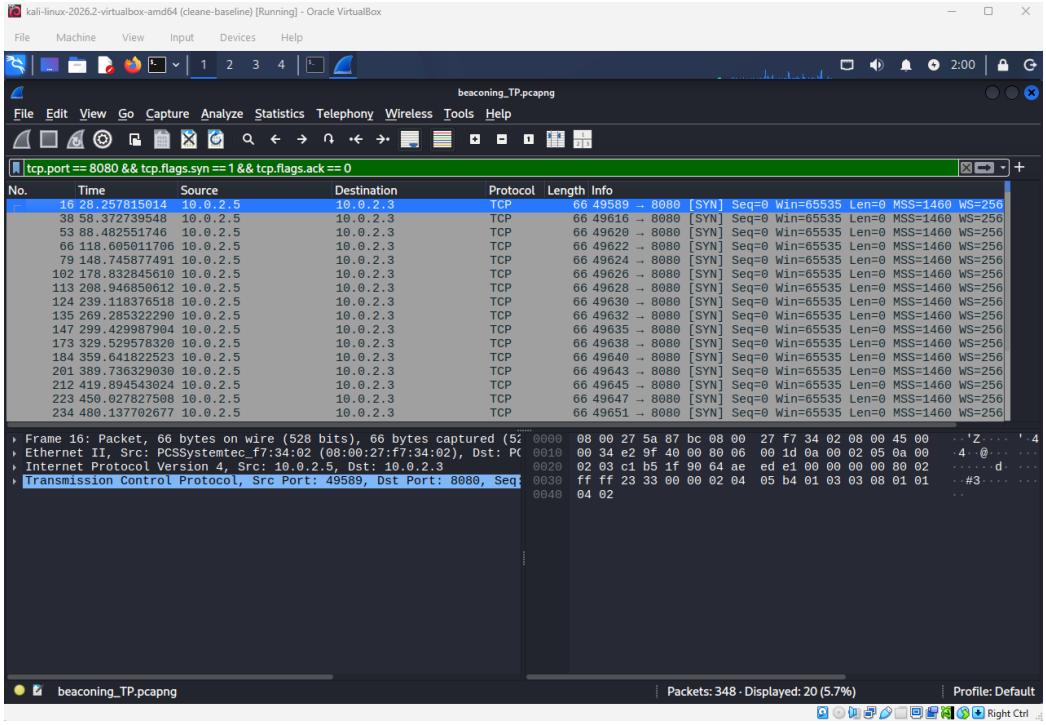
Sharp, evenly-spaced spikes — the visual signature of consistent beaconing.

Fig. 4 — Capture file properties



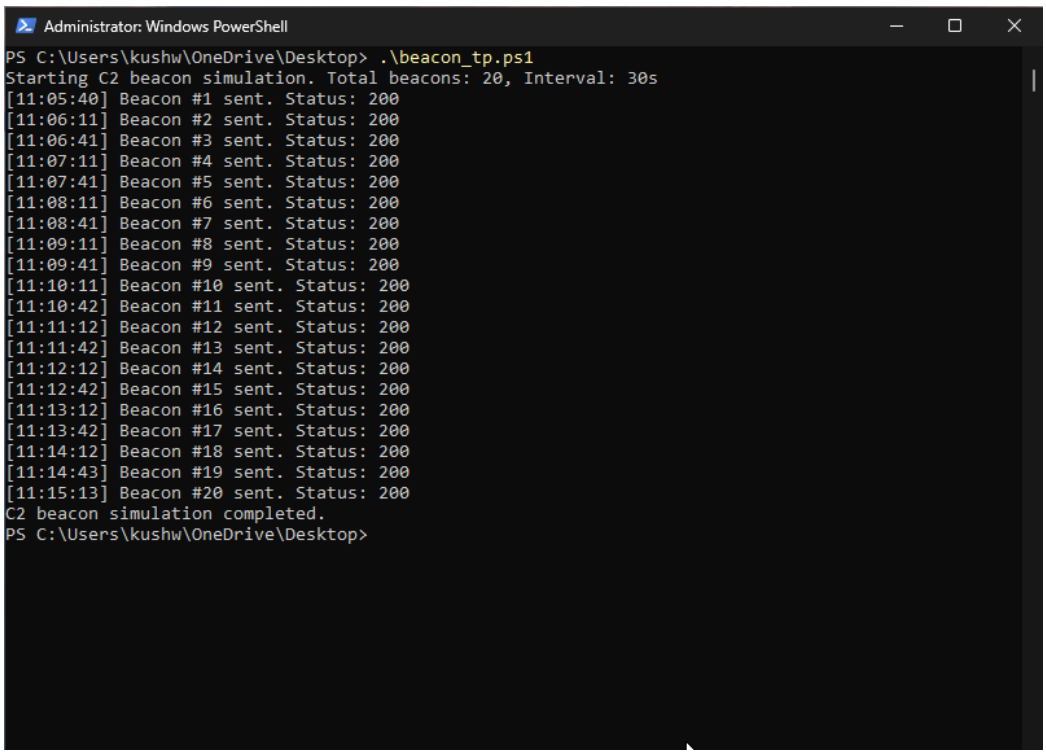
348 packets captured; filtered beacon traffic spans approximately 572 seconds (~9.5 minutes), consistent with 20 beacons at a 30-second interval.

Fig. 5 — Filtered SYN packets with timestamps



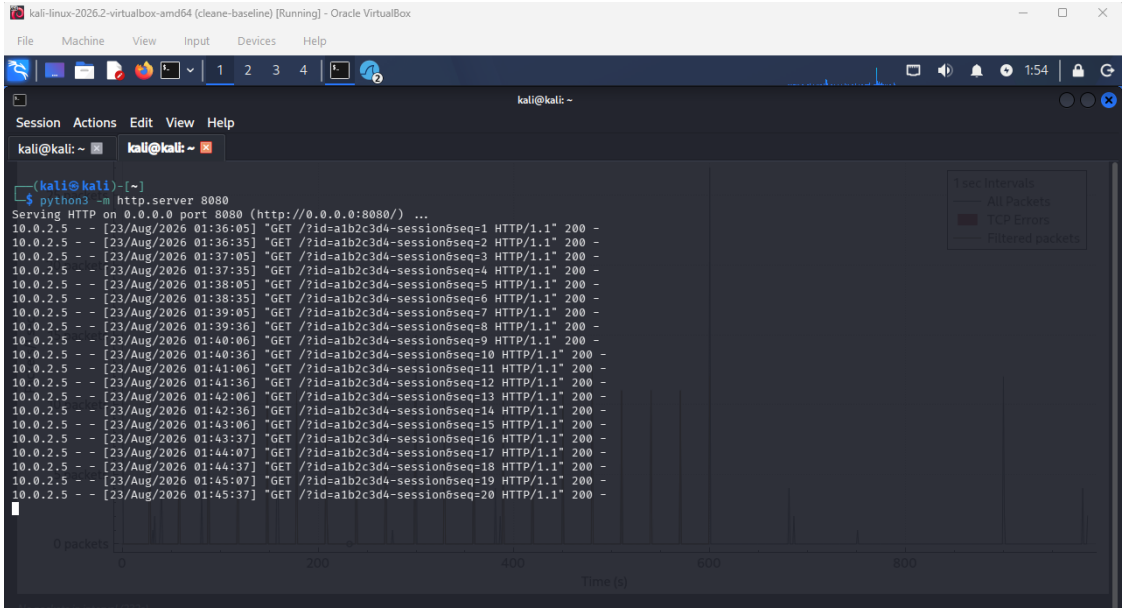
Isolated connection-attempt packets used to calculate the precise 30.09–30.17s interval range.

Fig. 6 — PowerShell script execution log



The script's own log of all 20 beacons sent, confirming the 30-second interval independently of the packet capture.

Fig. 7 — Kali HTTP server log



Server-side confirmation of all 20 GET requests received, with matching session-ID sequence numbers.

Beaconing — False Positive

Fig. 1 — Baseline capture, 0 packets

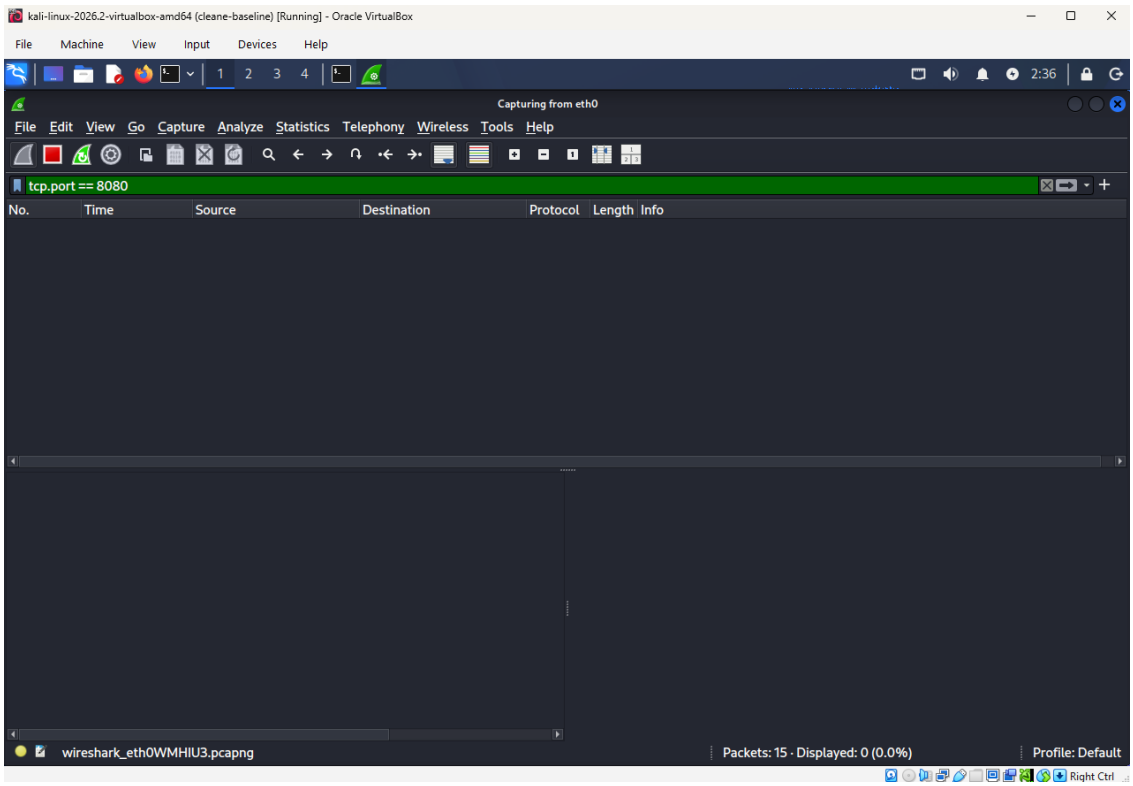
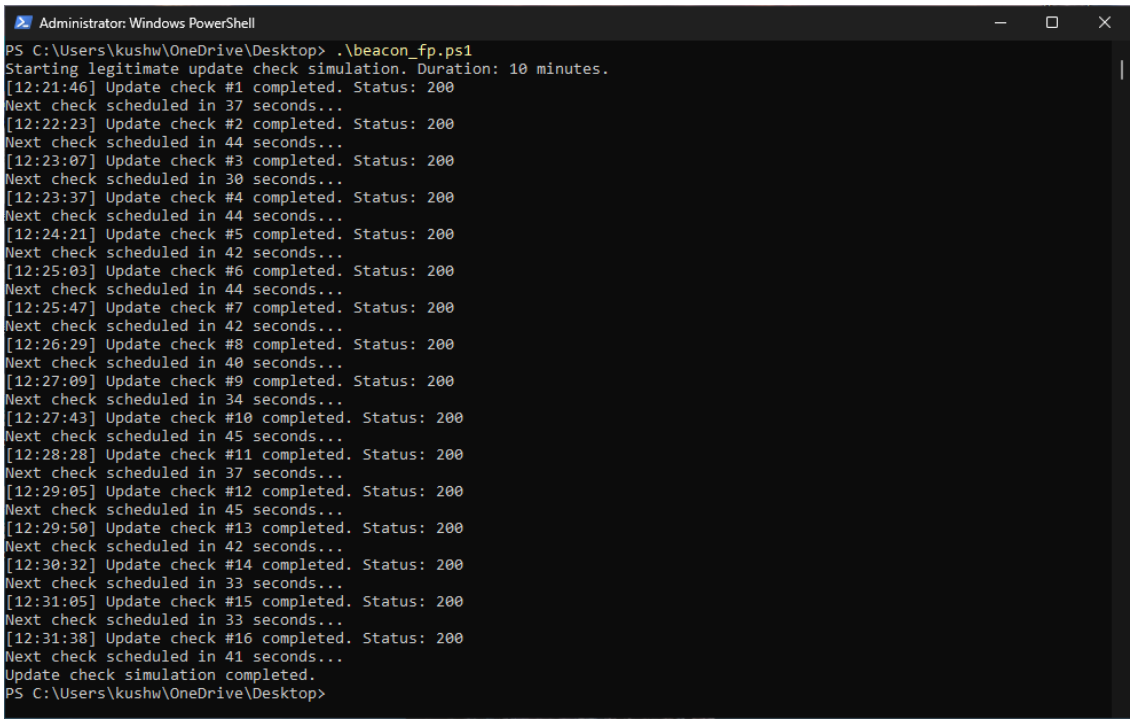
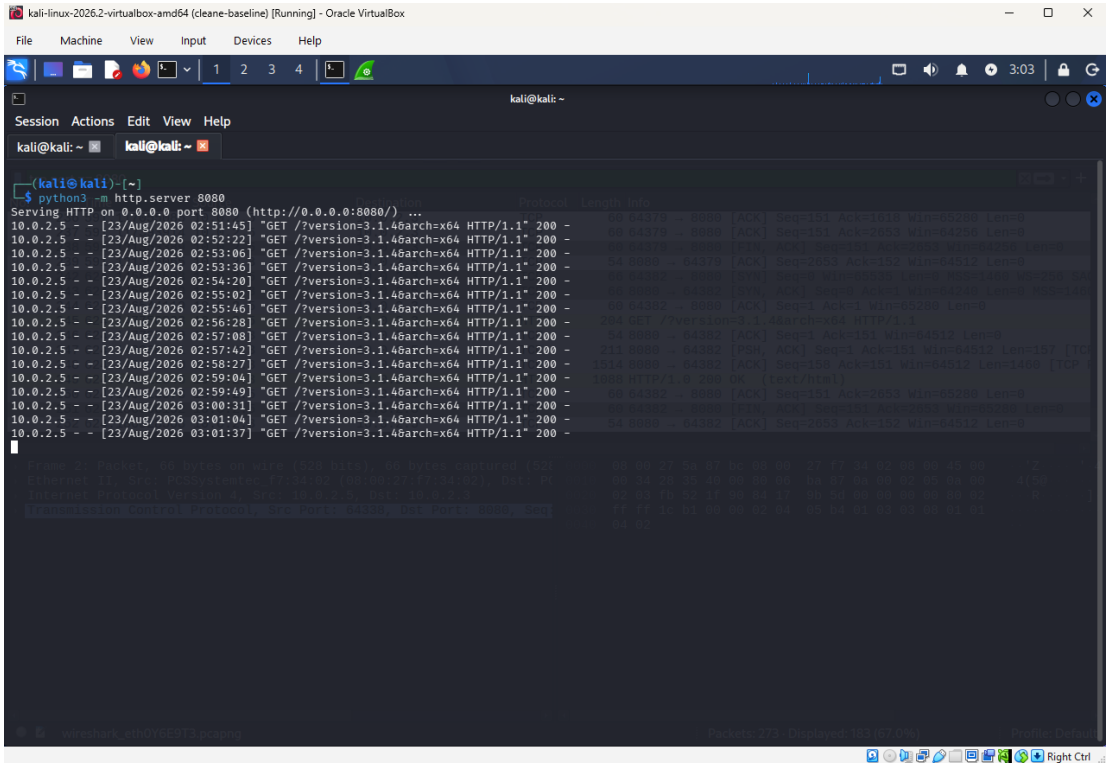


Fig. 2 — PowerShell script execution log



16 update checks completed, each logging its own randomized next-check delay (30–45 seconds).

Fig. 3 — Kali HTTP server log



Server-side confirmation of the same irregular request pattern, realistic version-check request path.

Fig. 4 — Full packet list

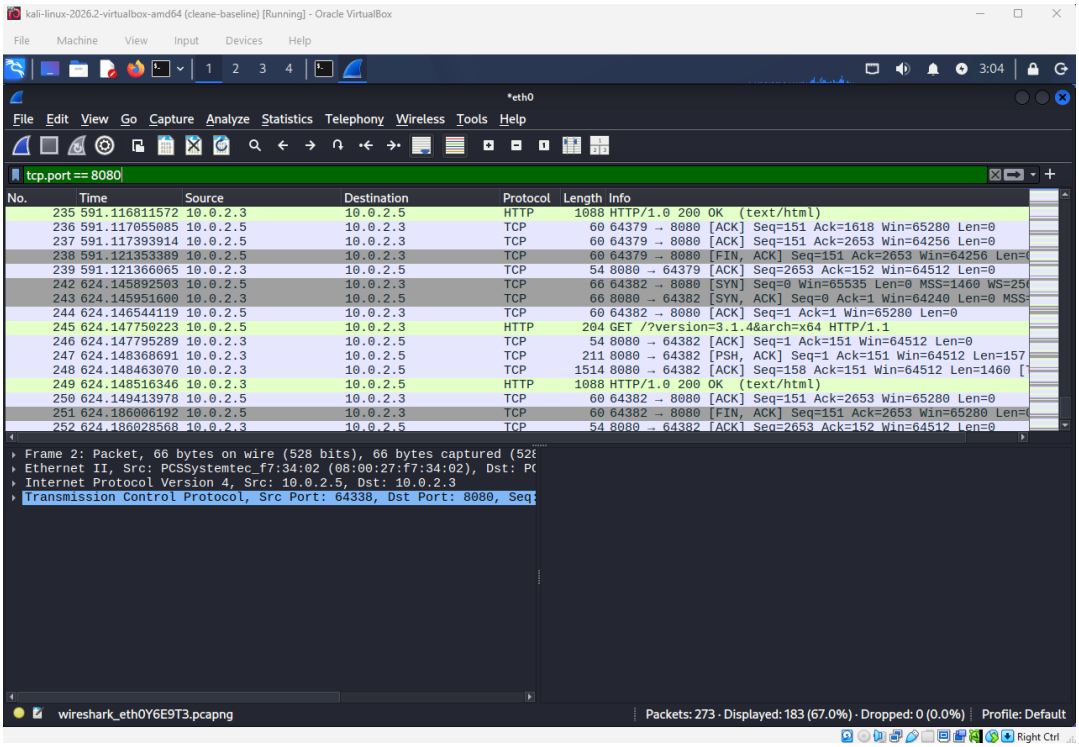
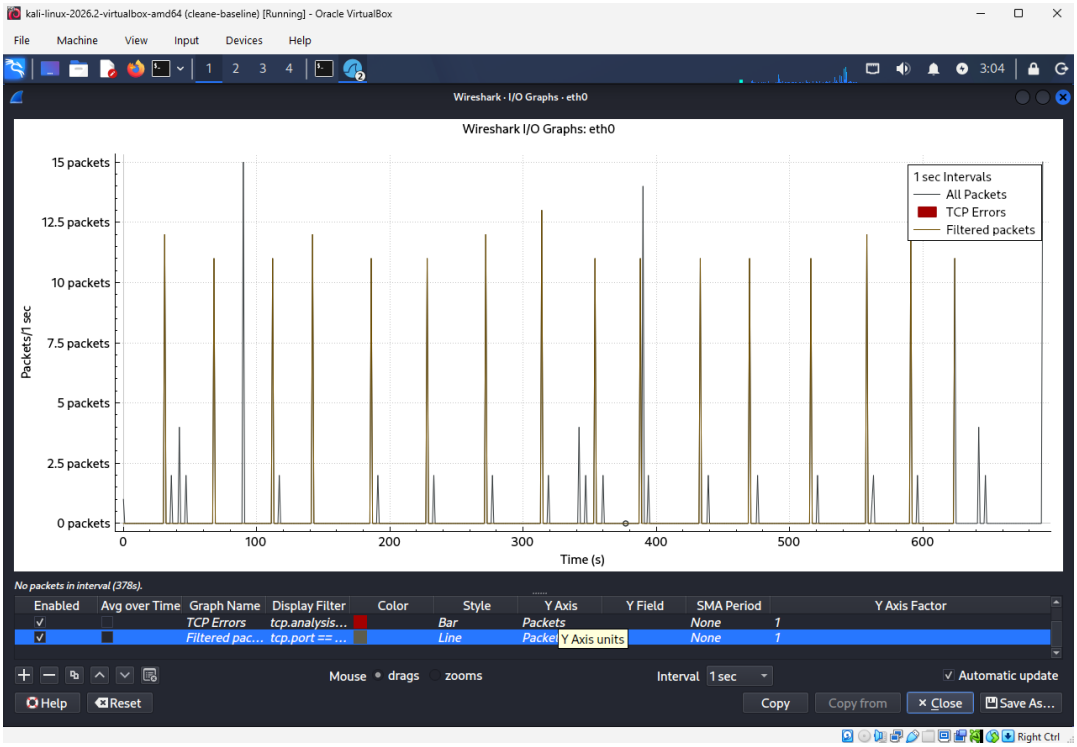
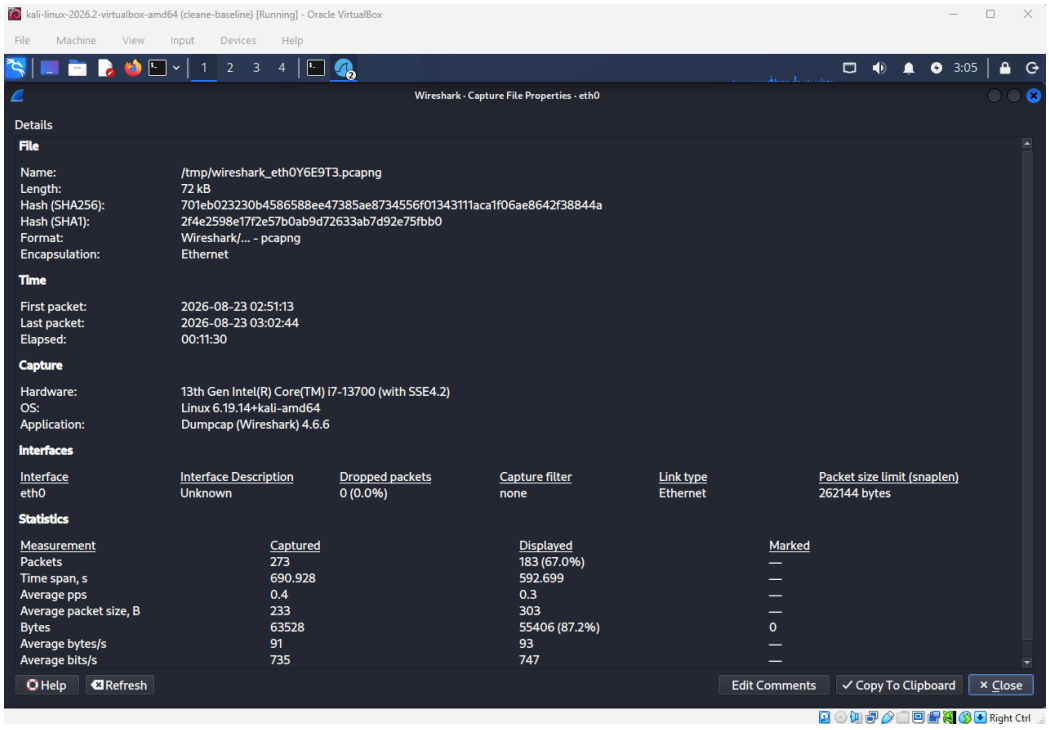


Fig. 5 — IO Graph, 1-second intervals



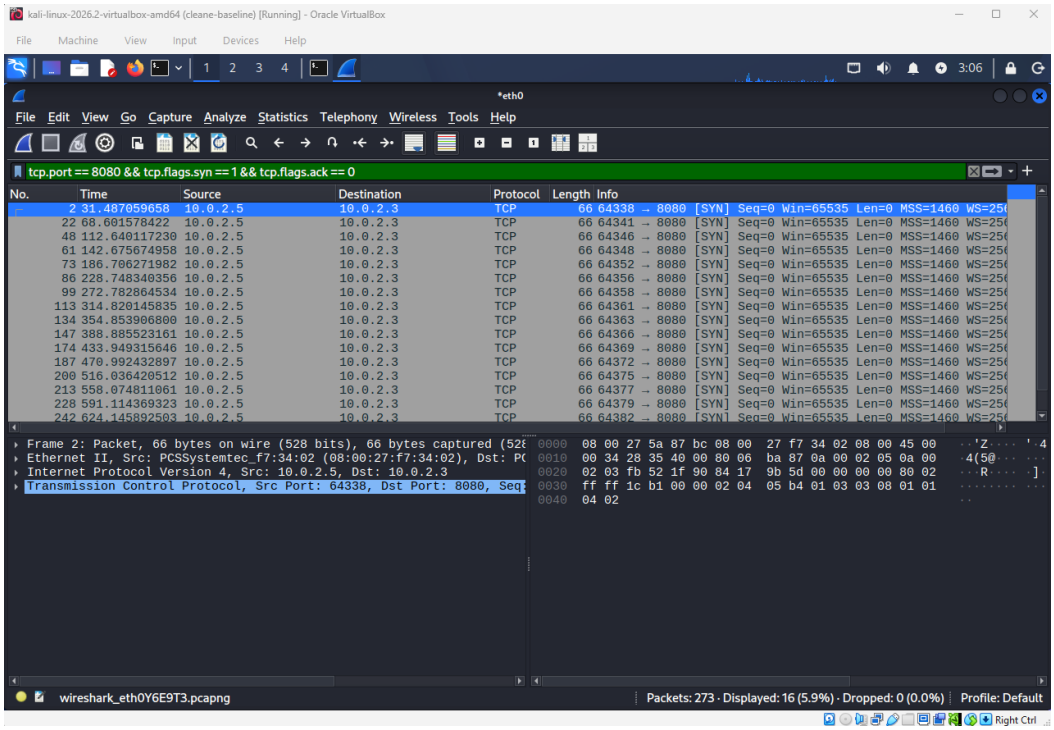
Visibly uneven spacing between spikes, contrasting with the true positive's regular pattern.

Fig. 6 — Capture file properties



273 packets captured; 11 minutes 30 seconds elapsed.

Fig. 7 — Filtered SYN packets with timestamps



Isolated connection-attempt packets used to calculate the 30–45s jittered interval range, matching the script's own logged values almost exactly.